

Taller de Lógica: Sincronización con Señales (IPC)

Malak Sanchez

Monitorías de Sistemas Operativos

14 de abril de 2026

Introducción

Este taller se enfoca en la coordinación asíncrona de procesos mediante el uso de señales POSIX (SIGUSR1, SIGUSR2). El objetivo es controlar el orden de ejecución de una jerarquía de procesos para que sigan una secuencia.

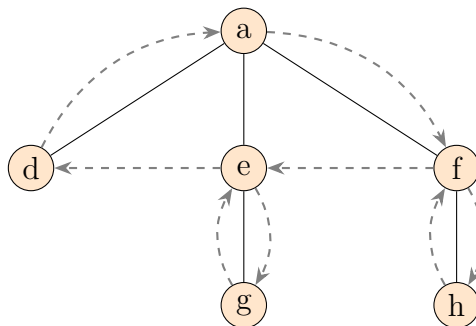
Bloque 1: De Diagramas a Código

Desarrolle el código necesario para replicar las siguientes estructuras y secuencias de señales. Todos los procesos deben imprimir su letra correspondiente al recibir o enviar el control.

Ejercicio 1

Replique la siguiente jerarquía. El flujo de señales debe iniciarse en el proceso raíz y viajar por las ramas de derecha a izquierda.

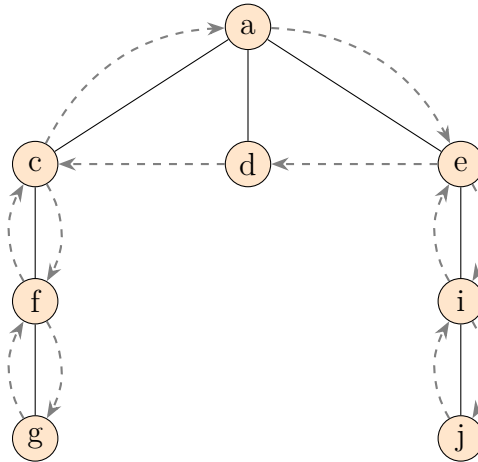
Secuencia sugerida: a-f-h-f-e-g-e-d-a



Ejercicio 2

Diseñe un programa que emule el siguiente flujo en cascada. Note que cada rama debe completarse antes de pasar al hermano de la izquierda.

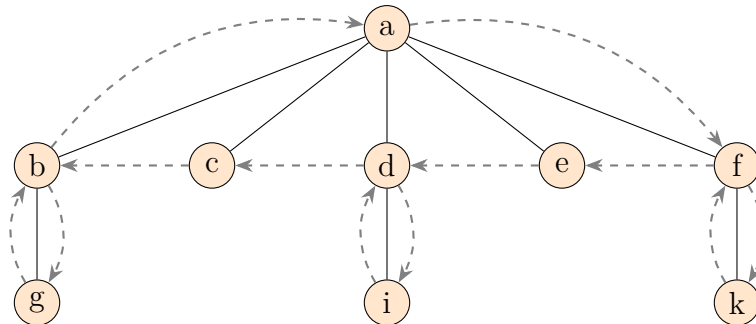
Secuencia sugerida: a-e-i-j-i-e-d-c-f-g-f-c-a



Ejercicio 3: El Serpentin

Implemente la sincronización para una jerarquía combinada. El flujo debe descender completamente por las ramas de la derecha antes de saltar a los hermanos de la izquierda. Note que algunos hermanos deben coordinarse directamente.

Secuencia sugerida: a-f-k-f-e-d-i-d-c-b-g-b-a



Bloque 2: Análisis de Código

Determine el orden de impresión de los siguientes fragmentos. Considere que el proceso padre siempre tiene el control inicial.

Ejercicio 4

Analice el siguiente código y determine la salida exacta por consola. Asuma que `fork()` tiene éxito y que el proceso padre siempre inicia la ejecución.

```
1 void handler(int sig) { /* Despierta el proceso */ }
2
3 int main(){
4     signal(SIGUSR1, handler);
5     pid_t p1, p2;
6     p1 = fork();
7     if(p1 == 0){
8         pause();
9         printf("H1\n");
10        kill(getppid(), SIGUSR1);
11        exit(0);
12    }
13    p2 = fork();
14    if(p2 == 0){
15        printf("H2\n");
16        kill(getppid(), SIGUSR1);
17        exit(0);
18    }
19    pause();
20    kill(p1, SIGUSR1);
21    pause();
22    printf("Padre\n");
23    return 0;
24 }
```